

ZPRAVODAJSKÝ PORTÁL

s AI p[ro]episem [a]lánků

BBC Design · Node.js Backend · Claude AI

RSS Scraping z 5 českých portálů

Automatický AI p[ro]epis každých 15 minut

PROJEKT	TECHNOLOGIE	VÝSTUP
news-portal/	Node.js + React + SQLite	VS Code projekt

OBSAH DOKUMENTU

1	Přehled projektu a architektura	3
2	Design: BBC vs CNN — analýza a rozhodnutí	4
3	Top 5 českých portálů — zdroje dat	5
4	Struktura projektu (souborový strom)	6
5	Backend — kompletní implementace	7
5.1	config.js — RSS zdroje a nastavení	7
5.2	database.js — SQLite schéma	8
5.3	rss-fetcher.js — stahování RSS	10
5.4	scraper.js — scraping fulltextu	11
5.5	ai-rewriter.js — Claude AI příspěvis	12
5.6	scheduler.js — cron pipeline	13
5.7	server.js — Express API	14
5.8	utils.js — pomocné funkce	15
6	Frontend — BBC design + React	16
6.1	global.css — kompletní styly	16
6.2	App.jsx — SPA router	20
6.3	HomePage.jsx	21
6.4	ArticleCard.jsx	22
6.5	SectionBlock.jsx	23
6.6	ArticlePage.jsx	24
6.7	utils.js + config.js	25
7	Konfigurace (package.json, vite, .env)	26
8	Instalace a spuštění	27
9	Právní poznámky	28

1. PŘEHLED PROJEKTU A ARCHITEKTURA

Tento dokument obsahuje kompletní zadání pro vývoj zpravodajského portálu ve VS Code. Portál automaticky agreguje zprávy z 5 nejvýznamnějších českých zpravodajských webů pomocí RSS, přepisuje je pomocí Claude AI a zobrazuje v prémiovém BBC-style designu.

Jak to funguje — pipeline

Krok	Modul	Co dělá	Interval
1	rss-fetcher.js	Stáhne RSS ze 5 zdrojů, uloží nové články do SQLite	každých 15 min
2	scraper.js	Pro každý nový článek scrapuje fulltext pomocí cheerio	ihned po RSS
3	ai-rewriter.js	Každý článek pošle do Claude API, uloží přepsanou verzi	ihned po scraping
4	server.js	Express API servíruje data frontendu na /api/*	stále běží
5	React frontend	BBC-style SPA zobrazuje přepsané články	při každém requestu

Technologický stack

Vrstva	Technologie	Balíček / verze
HTTP server	Express.js	express ^4.18
Databáze	SQLite	better-sqlite3 ^9.4
RSS parsing	RSS Parser	rss-parser ^3.13
Scraping	Cheerio	cheerio ^1.0 + axios ^1.6
AI přepis	Claude API	@anthropic-ai/sdk ^0.51
Scheduler	node-cron	node-cron ^3.0
Frontend	React + Vite	react ^18.2, vite ^5.0
Hosting	VPS / Vercel+Railway	—

2. DESIGN: BBC vs CNN — ANALÝZA

Kritérium	BBC.com	CNN.com
Styl	Editorský, seriózní, ■istý	Dramatický, video-first, hustý
Barvy	■ervená + bílá + ■erná	■ervená + tmavá + dynamická
Typografie	Reith Serif / Reith Sans	CNN Sans (vlastní)
Layout	Whitespace, lineární sekce	P■epln■ný grid, breaking bannery
Obrázky	16:9, velkorysé, editorské	16:9 + 9:16 pro video carousels
Labely	Minimální, diskrétní	LIVE / ANALYSIS / VIDEO všude
Mobile	Elegantní, ■itelné	Kompaktní, scroll-heavy
Vhodnost CZ	★★★★★ — Ideální	★★★██ — P■íliš americký

■ ROZHODNUTÍ: Použijeme BBC architekturu + BBC typografii + CNN breaking labely (LIVE/BREAKING ■ervené badgy).

BBC Design systém — klí■ové hodnoty

```
:root {
  --bbc-red:      #BB1919;    /* hlavní akcent */
  --bbc-red-dark: #990000;    /* hover stav */
  --bbc-black:   #1A1A1A;    /* navigace, header */
  --bbc-dark-grey: #404040;   /* sekundární text */
  --bbc-mid-grey: #6E6E6E;    /* meta, popisky */
  --bbc-light-grey: #EEEEEE;  /* odd■lova■e */
  --bbc-off-white: #F6F6F6;   /* pozadí stránky */
  --bbc-white:    #FFFFFF;     /* karty */
  --bbc-blue:     #0062B3;    /* linky, breadcrumby */

  /* Fonty */
  --font-serif: 'Playfair Display', Georgia, serif; /* titulky ■lánk■ */
  --font-sans: 'Source Sans 3', Arial, sans-serif; /* UI, navigace */
}
```

3. TOP 5 ČESKÝCH PORTÁLŮ — ZDROJE

Data dle NetMonitor SPIR-Gemius, průměrné měsíční reálné uživatele za rok 2024–2025:

#	Portál	Měs. uživatelé	RSS URL	Vlastník
1	Novinky.cz	5 mil.+	novinky.cz/rss2/	Seznam.cz / Borgis
2	Seznam Zprávy	4+ mil.	zpravy.seznam.cz/rss	Seznam.cz
3	iDnes.cz	3+ mil.	servis.idnes.cz/rss.aspx?c=zpravodaj	Mafra (Tykač)
4	Deník.cz	2+ mil.	denik.cz/rss/zpravy.html	Vltava Labe Media
5	Aktuálně.cz	~2 mil.	zpravy.aktualne.cz/rss/	Economia (Bakala)

■ RSS čtení je plně legální — weby ho publikují pro přesně tento účel. Scraping fulltextu je šedá zóna — vždy fallback na RSS perex pokud scraping selže.

Strategie získávání dat

Varianta	Metoda	Právní riziko	Doporučení
A — Bezpečná	RSS perex → Claude rozepíše	Žádné	■ Pro start
B — Plná	RSS + scraping fulltextu → pěepis	Střední	■ S opatrností
C — Hybridní	RSS perex + Claude komentář + odkaz	Žádné	■ Nejlepší UX

4. STRUKTURA PROJEKTU

```
news-portal/
├── backend/
│   ├── server.js           ← Express API server (port 3001)
│   ├── scheduler.js        ← node-cron pipeline (každých 15 min)
│   ├── rss-fetcher.js      ← stahování RSS z 5 zdrojů
│   ├── scraper.js          ← scraping fulltextu (axios + cheerio)
│   ├── ai-rewriter.js       ← Claude API píšící články
│   ├── database.js         ← SQLite schéma + všechny queries
│   ├── utils.js            ← generateSlug, guessCategory
│   └── config.js           ← RSS zdroje, AI config, kategorie
├── frontend/
│   ├── index.html
│   ├── vite.config.js
│   ├── package.json
│   └── src/
│       ├── main.jsx
│       ├── App.jsx         ← SPA hash router
│       ├── config.js       ← CATEGORIES konstanty
│       ├── utils.js        ← formatTimeAgo, formatDate
│       └── components/
│           ├── Header.jsx   ← logo + search + live nas
│           ├── NavBar.jsx   ← horizontální nav
│           ├── ArticleCard.jsx ← 3 typy: large/medium/small
│           ├── SectionBlock.jsx ← sekční blok (header + grid)
│           ├── BreakingBanner.jsx ← červený breaking banner
│           ├── Sidebar.jsx  ← nejčtenější + nejnovější
│           └── Footer.jsx   ← tmavý footer BBC styl
│       ├── pages/
│           ├── HomePage.jsx ← hero + sekce + sidebar
│           ├── ArticlePage.jsx ← detail článku + related
│           ├── CategoryPage.jsx ← výpis dle kategorie
│           └── SearchPage.jsx ← full-text hledání
│       ├── styles/
│           └── global.css   ← kompletní BBC styly
├── data/                  ← SQLite databáze (vytvorí: mkdir data)
│   └── news.db            ← automaticky vytvořeno
├── package.json          ← root dependencies
├── .env                  ← ANTHROPIC_API_KEY (neverzovat!)
├── .env.example
└── README.md
```

5. BACKEND — KOMPLETNÍ IMPLEMENTACE

5.1 backend/config.js

```

export const RSS_SOURCES = [
  {
    id: 'novinky',
    name: 'Novinky.cz',
    rssUrl: 'https://www.novinky.cz/rss2/',
    baseUrl: 'https://www.novinky.cz',
    articleSelector: '.article-body, .article-text, [class*="body"]',
    enabled: true
  },
  {
    id: 'idnes',
    name: 'iDnes.cz',
    rssUrl: 'https://servis.idnes.cz/rss.aspx?c=zpravodaj',
    baseUrl: 'https://www.idnes.cz',
    articleSelector: '.art-full-text, .article-body, [itemprop="articleBody"]',
    enabled: true
  },
  {
    id: 'aktualne',
    name: 'Aktuální.cz',
    rssUrl: 'https://zpravy.aktualne.cz/rss/',
    baseUrl: 'https://zpravy.aktualne.cz',
    articleSelector: '.article-body, .clanek-text',
    enabled: true
  },
  {
    id: 'denik',
    name: 'Deník.cz',
    rssUrl: 'https://www.denik.cz/rss/zpravy.html',
    baseUrl: 'https://www.denik.cz',
    articleSelector: '.article-body',
    enabled: true
  },
  {
    id: 'ct24',
    name: 'ČT24',
    rssUrl: 'https://ct24.ceskatelevize.cz/rss/hlavni-zpravy',
    baseUrl: 'https://ct24.ceskatelevize.cz',
    articleSelector: '.article__body, .article-body',
    enabled: true
  }
];

export const CATEGORIES = [
  { id: 'domaci', label: 'Domáci' },
  { id: 'zahranicni', label: 'Zahraničí' },
  { id: 'ekonomika', label: 'Ekonomika' },
  { id: 'sport', label: 'Sport' },
  { id: 'kultura', label: 'Kultura' },
  { id: 'technologie', label: 'Technologie' },
  { id: 'veda', label: 'Věda' },
  { id: 'zdravi', label: 'Zdraví' },
  { id: 'cestovani', label: 'Cestování' },
  { id: 'krimi', label: 'Krimi' }
];

export const AI_CONFIG = {
  model: 'claude-sonnet-4-20250514',
  maxTokens: 1500,
  rewriteDelayMs: 2000, // pauza mezi AI voláními (rate limiting)
  fetchIntervalMinutes: 15,
  maxArticlesPerFetch: 10, // max nových článků per run
};

```


5.2 backend/database.js — SQLite schéma

```
import Database from 'better-sqlite3';
import path from 'path';
import { fileURLToPath } from 'url';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const DB_PATH = path.join(__dirname, '../data/news.db');
let db;

export function getDb() {
  if (!db) { db = new Database(DB_PATH); db.pragma('journal_mode = WAL'); initSchema(); }
  return db;
}

function initSchema() {
  db.exec(`
    CREATE TABLE IF NOT EXISTS articles (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      source_id TEXT NOT NULL,          -- 'novinky' | 'idnes' | ...
      source_name TEXT NOT NULL,
      original_url TEXT UNIQUE NOT NULL,
      original_title TEXT NOT NULL,
      original_perex TEXT,
      original_body TEXT,              -- fulltext ze scrapingu

      rewritten_title TEXT,            -- výstup Claude AI
      rewritten_perex TEXT,
      rewritten_body TEXT,            -- JSON array of paragraphs

      category TEXT DEFAULT 'domaci',
      image_url TEXT,
      author TEXT,
      tags TEXT,                      -- JSON array

      published_at TEXT NOT NULL,
      fetched_at TEXT NOT NULL,
      rewritten_at TEXT,

      is_breaking INTEGER DEFAULT 0,
      views INTEGER DEFAULT 0,
      rewrite_status TEXT DEFAULT 'pending', -- pending|done|failed
      slug TEXT UNIQUE
    );

    CREATE INDEX IF NOT EXISTS idx_articles_published ON articles(published_at DESC);
    CREATE INDEX IF NOT EXISTS idx_articles_category ON articles(category);
    CREATE INDEX IF NOT EXISTS idx_articles_status ON articles(rewrite_status);

    CREATE TABLE IF NOT EXISTS fetch_log (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      source_id TEXT NOT NULL,
      fetched_at TEXT NOT NULL,
      articles_found INTEGER DEFAULT 0,
      articles_new INTEGER DEFAULT 0,
      error TEXT
    );
  `);
}
```

```
// ■■■ CRUD funckce ■■■
export function insertArticle(article) {
  return getDb().prepare(`
    INSERT OR IGNORE INTO articles
    (source_id, source_name, original_url, original_title, original_perex,
    original_body, category, image_url, author, tags, published_at,
    fetched_at, rewrite_status, slug, is_breaking)
    VALUES
    (@source_id, @source_name, @original_url, @original_title, @original_perex,
    @original_body, @category, @image_url, @author, @tags, @published_at,
    @fetched_at, @rewrite_status, @slug, @is_breaking)
  `).run(article);
}

export function updateRewrittenArticle(id, data) {
  return getDb().prepare(`
    UPDATE articles
    SET rewritten_title=@title, rewritten_perex=@perex,
    rewritten_body=@body, rewrite_status='done', rewritten_at=@rewritten_at
    WHERE id=@id
  `).run({ ...data, id });
}

export function getPendingRewrites(limit = 10) {
  return getDb().prepare(`
    SELECT * FROM articles
    WHERE rewrite_status='pending' AND original_body IS NOT NULL
    ORDER BY published_at DESC LIMIT ?
  `).all(limit);
}

export function getArticles({ category, source, limit=20, offset=0, search={} }) {
  let where = "WHERE rewrite_status='done'";
  const params = [];
  if (category) { where += ' AND category=?'; params.push(category); }
  if (source) { where += ' AND source_id=?'; params.push(source); }
  if (search) { where += ' AND (rewritten_title LIKE ? OR rewritten_perex LIKE ?)';
    params.push(`%${search}%`, `%${search}%`); }
  return getDb().prepare(
    `SELECT * FROM articles ${where} ORDER BY published_at DESC LIMIT ? OFFSET ?`
  ).all(...params, limit, offset);
}

export function getArticleBySlug(slug) {
  getDb().prepare(`UPDATE articles SET views=views+1 WHERE slug=?`).run(slug);
  return getDb().prepare(`SELECT * FROM articles WHERE slug=?`).get(slug);
}

export function getLatestArticles(limit=50) {
  return getDb().prepare(
    `SELECT * FROM articles WHERE rewrite_status='done' ORDER BY published_at DESC LIMIT ?`
  ).all(limit);
}

export function getStats() {
  const db = getDb();
  return {
    total: db.prepare(`SELECT COUNT(*) as c FROM articles`).get().c,
    done: db.prepare(`SELECT COUNT(*) as c FROM articles WHERE rewrite_status='done'`).get().c,
    pending: db.prepare(`SELECT COUNT(*) as c FROM articles WHERE rewrite_status='pending'`).get().c,
    failed: db.prepare(`SELECT COUNT(*) as c FROM articles WHERE rewrite_status='failed'`).get().c,
  };
}
```

5.3 backend/rss-fetcher.js

```

import Parser from 'rss-parser';
import { RSS_SOURCES } from './config.js';
import { insertArticle } from './database.js';
import { generateSlug, guessCategory } from './utils.js';

const parser = new Parser({
  timeout: 10000,
  headers: { 'User-Agent': 'Mozilla/5.0 (compatible; NewsBot/1.0)' }
});

export async function fetchAllFeeds() {
  const results = [];
  for (const source of RSS_SOURCES.filter(s => s.enabled)) {
    try {
      const result = await fetchFeed(source);
      results.push(result);
      console.log(`[RSS] ${source.name}: ${result.newCount} nových článků`);
    } catch (err) {
      console.error(`[RSS] Chyba ${source.name}:`, err.message);
    }
    await sleep(1000); // ohleduplná pauza mezi zdroji
  }
  return results;
}

async function fetchFeed(source) {
  const feed = await parser.parseURL(source.rssUrl);
  let newCount = 0;

  for (const item of feed.items.slice(0, 20)) {
    const article = {
      source_id: source.id,
      source_name: source.name,
      original_url: item.link || item.guid,
      original_title: item.title || 'Bez titulu',
      original_perex: stripHtml(item.contentSnippet || item.summary || ''),
      original_body: null, // doplní scraper
      category: guessCategory(item.title, item.categories || []),
      image_url: extractImage(item),
      author: item.author || item.creator || source.name,
      tags: JSON.stringify(item.categories || []),
      published_at: item.pubDate ? new Date(item.pubDate).toISOString()
        : new Date().toISOString(),
      fetched_at: new Date().toISOString(),
      rewrite_status: 'pending',
      slug: generateSlug(item.title || '', item.link || ''),
      is_breaking: isBreaking(item.title) ? 1 : 0
    };
    const result = insertArticle(article);
    if (result.changes > 0) newCount++;
  }
  return { source: source.id, newCount };
}

function extractImage(item) {
  if (item.enclosure?.url) return item.enclosure.url;
  if (item['media:content']?.['$']?.url) return item['media:content']['$'].url;
  const m = (item.content || '').match(/<img[^>]+src="([^"]+)"/);
  return m ? m[1] : null;
}

function isBreaking(title = '') {
  const kw = ['zemětřesení', 'exploze', 'útok', 'neštěstí', 'tragédie', 'havárie', 'požár'];
  return kw.some(k => title.toLowerCase().includes(k));
}

function stripHtml(html) {
  return html.replace(/<[^>]*>/g, '').replace(/&[a-z]+;/g, ' ').trim();
}

function sleep(ms) { return new Promise(r => setTimeout(r, ms)); }

```

5.4 backend/scrapper.js

```
import axios from 'axios';
import * as cheerio from 'cheerio';
import { RSS_SOURCES } from './config.js';
import { getDb } from './database.js';

const http = axios.create({
  timeout: 15000,
  headers: {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36',
    'Accept': 'text/html,application/xhtml+xml',
    'Accept-Language': 'cs-CZ,cs;q=0.9',
  }
});

export async function scrapeArticleBodies() {
  const db = getDb();
  const articles = db.prepare(`
    SELECT id, original_url, source_id FROM articles
    WHERE original_body IS NULL AND rewrite_status='pending'
    ORDER BY published_at DESC LIMIT 15
  `).all();

  console.log(`[SCRAPER] Scrapuji fulltext pro ${articles.length} článků`);

  for (const article of articles) {
    try {
      const body = await scrapeArticle(article.original_url, article.source_id);
      if (body && body.length > 100) {
        db.prepare(`UPDATE articles SET original_body=? WHERE id=?`).run(body, article.id);
      } else {
        // fallback: použij perex jako základ pro příběh
        db.prepare(`UPDATE articles SET original_body=original_perex WHERE id=?`)
          .run(article.id);
      }
    } catch (err) {
      console.error(`[SCRAPER] Chyba ${article.original_url}:`, err.message);
      db.prepare(`UPDATE articles SET original_body=original_perex WHERE id=?`)
        .run(article.id);
    }
    await sleep(2000); // 2s pauza – ohleduplnost k cílovým serverům
  }
}

async function scrapeArticle(url, sourceId) {
  const source = RSS_SOURCES.find(s => s.id === sourceId);
  const selector = source?.articleSelector || 'article, .article-body, main p';
  const response = await http.get(url);
  const $ = cheerio.load(response.data);
  // odstraní šum
  $('script,style,nav,header,footer,.ad,.advertisement,.related,.comments').remove();
  // zkus specifický selector
  let text = $(selector).text().trim();
  // fallback
  if (!text || text.length < 100) {
    text = $('article').text().trim() || $('main').text().trim();
  }
  return text.replace(/\s+/g, ' ').replace(/\n{3,}/g, '\n\n').trim().substring(0, 8000);
}

function sleep(ms) { return new Promise(r => setTimeout(r, ms)); }
```

5.5 backend/ai-rewriter.js — Claude API

```

import Anthropic from '@anthropic-ai/sdk';
import { getPendingRewrites, updateRewrittenArticle, markRewriteFailed } from './database.js';
import { AI_CONFIG } from './config.js';

const anthropic = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

const SYSTEM_PROMPT = `Jsi redaktor českého zpravodajského portálu. Dostaneš lánek
z jiného média a tvým úkolem je ho kompletně přepsat vlastními slovy.

PRAVIDLA:
- Zachovej všechna fakta, čísla, jména a data přesně
- Kompletně změň formulace, strukturu vět a pořadí informací
- Piš přirozenou českou novinářskou češtinou
- Titulek musí být výstižný, max 100 znaků
- Perex 1-2 věty, max 200 znaků
- Body: 3-5 odstavců, každý 3-5 vět

VÝSTUP: Odpověz POUZE validním JSON, nic jiného:
{
  "title": "Přepsaný titulek",
  "perex": "Přepsaný perex.",
  "body": ["První odstavec...", "Druhý odstavec...", "Třetí odstavec..."]
};

export async function rewritePendingArticles() {
  const articles = getPendingRewrites(AI_CONFIG.maxArticlesPerFetch);
  console.log(`[AI] Přepisuji ${articles.length} článků`);

  for (const article of articles) {
    try {
      const rewritten = await rewriteArticle(article);
      updateRewrittenArticle(article.id, {
        title: rewritten.title,
        perex: rewritten.perex,
        body: JSON.stringify(rewritten.body),
        rewritten_at: new Date().toISOString()
      });
      console.log(`[AI] ✓ Přepsán: ${rewritten.title.substring(0, 60)}...`);
    } catch (err) {
      console.error(`[AI] ✗ ID ${article.id}:`, err.message);
      markRewriteFailed(article.id, err.message);
    }
    await sleep(AI_CONFIG.rewriteDelayMs); // rate limiting
  }
}

async function rewriteArticle(article) {
  const userPrompt = `Přepiš tento lánek:

TITULEK: ${article.original_title}
PEREX: ${article.original_perex || ''}
TEXT: ${article.original_body || article.original_perex || ''}`;

  const message = await anthropic.messages.create({
    model: AI_CONFIG.model,
    max_tokens: AI_CONFIG.maxTokens,
    system: SYSTEM_PROMPT,
    messages: [{ role: 'user', content: userPrompt }]
  });

  const raw = message.content[0].text.trim()
    .replace(/^\`\`\`json\s*/i, '').replace(/\`\`\`\s*$/, '').trim();

  const parsed = JSON.parse(raw);
  if (!parsed.title || !parsed.perex || !Array.isArray(parsed.body)) {
    throw new Error('Neplatný formát odpovědi AI');
  }
  return parsed;
}

function sleep(ms) { return new Promise(r => setTimeout(r, ms)); }

```

5.6 backend/scheduler.js

```
import cron from 'node-cron';
import { fetchAllFeeds } from './rss-fetcher.js';
import { scrapeArticleBodies } from './scraper.js';
import { rewritePendingArticles } from './ai-rewriter.js';

export function startScheduler() {
  console.log('[SCHEDULER] Scheduler spuštění');

  // Pipeline každých 15 minut
  cron.schedule('*/*15 * * * *', async () => {
    console.log(`\n[PIPELINE] Start ${new Date().toISOString()}`);
    try {
      await fetchAllFeeds(); // 1. RSS
      await scrapeArticleBodies(); // 2. Fulltext
      await rewritePendingArticles(); // 3. AI přepis
    } catch (err) {
      console.error('[PIPELINE] Chyba:', err);
    }
    console.log(`[PIPELINE] End\n`);
  });

  // První run po startu (po 5s)
  setTimeout(async () => {
    await fetchAllFeeds();
    await scrapeArticleBodies();
    await rewritePendingArticles();
  }, 5000);
}
```

5.7 backend/server.js — Express API

5.8 backend/utils.js

```
export function generateSlug(title, url) {
  const slug = title.toLowerCase()
    .normalize('NFD').replace(/[\u0300-\u036f]/g, '') // remove diacritics
    .replace(/[^a-z0-9\s-]/g, '')
    .replace(/\s+/g, '-').replace(/-+/g, '-')
    .substring(0, 80);
  const hash = Math.abs(url.split('').reduce((a,c) => ((a<<5)-a)+c.charCodeAt(0), 0))
    .toString(36).substring(0, 6);
  return `${slug}-${hash}`;
}

export function guessCategory(title = '', rssCategories = []) {
  const text = (title + ' ' + rssCategories.join(' ')).toLowerCase();
  const mapping = {
    domaci: ['esk', 'eská', 'prague', 'praha', 'vláda', 'snmovna', 'babiš', 'fiala'],
    zahranicni: ['zahrani', 'svt', 'nmecko', 'usa', 'rusko', 'ukraji', 'nato', 'eu'],
    ekonomika: ['ekonom', 'inflac', 'koruna', 'burza', 'banka', 'ceny', 'zdraž', 'firma'],
    sport: ['fotbal', 'hokej', 'tenis', 'sport', 'ms ', 'me ', 'liga', 'olymp'],
    kultura: ['kultura', 'film', 'seriál', 'hudba', 'divadl', 'výstav', 'kniha'],
    technologie: ['technolog', 'ai ', 'umlá inteligence', 'software', 'hardware', 'mobil'],
    veda: ['vda', 'výzkum', 'vesmír', 'biolog', 'klímat', 'fyzik', 'objev'],
    zdravi: ['zdraví', 'nemoc', 'léba', 'nemocn', 'lék', 'rakovina'],
    cestovani: ['cestov', 'dovolená', 'letišt', 'turistik', 'hotel'],
    krimi: ['policie', 'vražda', 'loupež', 'krimi', 'soud', 'útok', 'nehoda'],
  };
  for (const [cat, kw] of Object.entries(mapping)) {
    if (kw.some(k => text.includes(k))) return cat;
  }
  return 'domaci';
}
```

6. FRONTEND — BBC DESIGN + REACT

6.1 frontend/src/styles/global.css — Kompletní BBC styly

■ Kompletní CSS (cca 600 řádků) — zahrnuje BBC barevnou paletu, typografii, všechny typy karet, hero sekce, sidebar, detail článku, footer a responzivitu. Viz kompletní kód v předchozím chatu nebo níže klíčové části:

```
@import url('https://fonts.googleapis.com/css2?family=Playfair+Display:wght@400;700&family=Source+Sans+3:wght@400;500;600&display=swap');

:root {
  --bbc-red: #BB1919; --bbc-red-dark: #990000;
  --bbc-black: #1A1A1A; --bbc-dark-grey: #404040;
  --bbc-mid-grey: #6E6E6E; --bbc-light-grey: #EEEEEE;
  --bbc-off-white: #F6F6F6; --bbc-white: #FFFFFF;
  --bbc-blue: #0062B3;
  --font-serif: 'Playfair Display', Georgia, serif; /* titulky */
  --font-sans: 'Source Sans 3', Arial, sans-serif; /* UI */
  --max-width: 1280px; --sidebar-width: 280px;
}

/* HEADER — tmavý sticky, červená spodní linka */
.site-header {
  background: var(--bbc-black); padding: 16px 0;
  position: sticky; top: 0; z-index: 1000;
  border-bottom: 3px solid var(--bbc-red);
}

/* NAVIGACE — tmavá, horizontálně scrollovatelná */
.main-nav { background: var(--bbc-black); overflow-x: auto; scrollbar-width: none; }
.main-nav li a { padding: 12px 14px; color: #ccc; font-size: 14px;
  border-bottom: 3px solid transparent; transition: all 0.15s; }
.main-nav li a.active { color: white; border-bottom-color: var(--bbc-red); }

/* SEKCE HNĚDÁ — červená linka 3px */
.section-header { border-bottom: 3px solid var(--bbc-red); }
.section-header h2 { font-size: 20px; text-transform: uppercase; letter-spacing: 0.5px; }

/* ARTICLE CARD — hover efekt */
.article-card { transition: transform 0.15s ease, box-shadow 0.15s ease; }
.article-card:hover { transform: translateY(-2px); box-shadow: 0 4px 12px rgba(0,0,0,.1); }

/* LARGE CARD */
.article-card--large .card-title { font-size: 28px; font-family: var(--font-serif); }

/* MEDIUM CARD — grid s thumbnailem */
.article-card--medium {
  display: grid; grid-template-columns: 120px 1fr;
  gap: 16px; padding: 16px 0; border-top: 1px solid var(--bbc-light-grey);
}

/* BREAKING / LIVE badges (CNN style) */
.badge--breaking { background: #CC0000; color: white; font-size: 10px;
  font-weight: 700; text-transform: uppercase; padding: 2px 6px; }
.badge--live { background: #CC0000; color: white; }

/* LAYOUT */
.page-grid { display: grid; grid-template-columns: 1fr 280px; gap: 40px; }
.hero-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 24px; }

/* RESPONSIVE */
@media (max-width: 1024px) { .page-grid { grid-template-columns: 1fr; } }
@media (max-width: 768px) { .article-card--large .card-title { font-size: 22px; } }
```

6.2 frontend/src/App.jsx — SPA Hash Router

```

import { useState, useEffect } from 'react';
import Header from './components/Header.jsx';
import NavBar from './components/NavBar.jsx';
import BreakingBanner from './components/BreakingBanner.jsx';
import HomePage from './pages/HomePage.jsx';
import ArticlePage from './pages/ArticlePage.jsx';
import CategoryPage from './pages/CategoryPage.jsx';
import SearchPage from './pages/SearchPage.jsx';
import Footer from './components/Footer.jsx';

const API_BASE = import.meta.env.VITE_API_URL || 'http://localhost:3001';
export const api = (path) => fetch(`${API_BASE}${path}`).then(r => r.json());

export default function App() {
  const [route, setRoute] = useState(parseHash());
  const [stats, setStats] = useState(null);

  useEffect(() => {
    const onHash = () => setRoute(parseHash());
    window.addEventListener('hashchange', onHash);
    return () => window.removeEventListener('hashchange', onHash);
  }, []);

  useEffect(() => { api('/api/stats').then(setStats).catch(()=>{}); }, []);

  function navigate(path) { window.location.hash = path; }

  return (
    <div className="app">
      {stats && (
        <div className="stats-bar">
          Celkem: <span>{stats.total}</span> | AI p■epsáno: <span>{stats.done}</span>
          | ■eká: <span>{stats.pending}</span>
        </div>
      )}
      <Header navigate={navigate} />
      <NavBar route={route} navigate={navigate} />
      <BreakingBanner />
      <main>
        {route.page === 'home' && <HomePage navigate={navigate} />}
        {route.page === 'article' && <ArticlePage slug={route.slug} navigate={navigate} />}
        {route.page === 'category' && <CategoryPage category={route.category} navigate={navigate} />}
        {route.page === 'search' && <SearchPage query={route.query} navigate={navigate} />}
      </main>
      <Footer navigate={navigate} />
      <ScrollToTop />
    </div>
  );
}

function parseHash() {
  const hash = window.location.hash.replace('#', '') || '/';
  if (hash === '/') return { page: 'home' };
  if (hash.startsWith('/clanek/')) return { page: 'article', slug: hash.replace('/clanek/', '') };
  if (hash.startsWith('/kategorie/')) return { page: 'category', category: hash.replace('/kategorie/', '') };
  if (hash.startsWith('/hledani')) return { page: 'search', query: new URLSearchParams(hash.split('?')[1]).get('q') || '' };
  return { page: 'home' };
}

function ScrollToTop() {
  const [v, setV] = useState(false);
  useEffect(() => {
    const fn = () => setV(window.scrolly > 400);
    window.addEventListener('scroll', fn);
    return () => window.removeEventListener('scroll', fn);
  }, []);
  return <button className={`scroll-top ${v?'visible':''}`}
    onClick={() => window.scrollTo({top:0, behavior:'smooth'})}>↑</button>;
}

```

6.3 frontend/src/pages/HomePage.jsx

```
import { useState, useEffect } from 'react';
import { api } from '../App.jsx';
import ArticleCard from '../components/ArticleCard.jsx';
import SectionBlock from '../components/SectionBlock.jsx';
import Sidebar from '../components/Sidebar.jsx';
import { CATEGORIES } from '../config.js';

export default function HomePage({ navigate }) {
  const [articles, setArticles] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api('/api/articles/latest?limit=60')
      .then(data => { setArticles(data); setLoading(false); })
      .catch(() => setLoading(false));
  }, []);

  if (loading) return <HomeSkeleton />;

  const hero = articles[0];
  const secondary = articles.slice(1, 4);
  const rest = articles.slice(4);

  const byCategory = {};
  CATEGORIES.forEach(cat => {
    byCategory[cat.id] = rest.filter(a => a.category === cat.id).slice(0, 4);
  });

  return (
    <div className="container" style={{ padding: 32 }}>
      <div style={{ display: flex; justify-content: space-between; align-items: center; margin-bottom: 16px; border-bottom: 1px solid #ccc; padding-bottom: 16px; width: 100%; }}>
        <div style="width: 60%; text-align: center;">
          <h1 style="margin: 0; font-size: 2em; font-weight: normal;">Hero Section
            <div style="width: 45%; text-align: left;">
              <h2 style="margin: 0; font-size: 1.2em; font-weight: normal;">Secondary Section
                <div style="width: 48%;">
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
              <h2 style="margin: 0; font-size: 1.2em; font-weight: normal;">Secondary Section
                <div style="width: 48%;">
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
          <h1 style="margin: 0; font-size: 2em; font-weight: normal;">Hero Section
            <div style="width: 45%; text-align: left;">
              <h2 style="margin: 0; font-size: 1.2em; font-weight: normal;">Secondary Section
                <div style="width: 48%;">
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
              <h2 style="margin: 0; font-size: 1.2em; font-weight: normal;">Secondary Section
                <div style="width: 48%;">
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
        <div style="width: 60%;">
          <div style="display: flex; justify-content: space-between; margin-bottom: 10px;">
            <div style="width: 45%;">
              <h2 style="margin: 0; font-size: 1.2em; font-weight: normal;">Secondary Section
                <div style="width: 48%;">
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
              <h2 style="margin: 0; font-size: 1.2em; font-weight: normal;">Secondary Section
                <div style="width: 48%;">
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
            <div style="width: 45%;">
              <h2 style="margin: 0; font-size: 1.2em; font-weight: normal;">Secondary Section
                <div style="width: 48%;">
                  <h3 style="margin: 0; font-size: 1.1em; font-weight: normal;">Tertiary Section
                    <div style="width: 48%;">
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                      <h4 style="margin: 0; font-size: 1.05em; font-weight: normal;">Quaternary Section
                  <h3 style="margin: 0; font-size: 
```

6.4 frontend/src/components/ArticleCard.jsx

```
import { formatTimeAgo } from '../utils.js';

export default function ArticleCard({ article, size='medium', navigate }) {
  const href = `#/clanek/${article.slug}`;
  const go = e => { e.preventDefault(); navigate(`/clanek/${article.slug}`); };

  const Cat = () => <span className="card-category">{article.category?.toUpperCase()}</span>;
  const Meta = () => (
    <div className="card-meta">
      {article.isBreaking && <span className="badge badge--breaking">Breaking</span>}
      <span>{article.sourceName}</span><span>.</span>
      <span>{formatTimeAgo(article.publishedAt)}</span>
    </div>
  );

  if (size === 'large') return (
    <article className="article-card article-card--large">
      {article.imageUrl && (
        <a href={href} onClick={go} className="card-img">
          <img src={article.imageUrl} alt={article.title} loading="lazy"
            onError={e => e.target.style.display='none'} />
        </a>
      )}
      <div className="card-body">
        <Cat />
        <h2 className="card-title"><a href={href} onClick={go}>{article.title}</a></h2>
        {article.perex && <p className="card-perex">{article.perex}</p>}
        <Meta />
      </div>
    </article>
  );

  if (size === 'medium') return (
    <article className="article-card article-card--medium">
      {article.imageUrl && (
        <a href={href} onClick={go} className="card-img">
          <img src={article.imageUrl} alt={article.title} loading="lazy"
            onError={e => e.target.style.display='none'} />
        </a>
      )}
      <div className="card-body">
        <Cat />
        <h3 className="card-title"><a href={href} onClick={go}>{article.title}</a></h3>
        <Meta />
      </div>
    </article>
  );

  return ( // small
    <article className="article-card article-card--small">
      <Cat />
      <h3 className="card-title"><a href={href} onClick={go}>{article.title}</a></h3>
      <Meta />
    </article>
  );
}
```

6.5 frontend/src/components/SectionBlock.jsx

```
import ArticleCard from '../ArticleCard.jsx';

export default function SectionBlock({ title, categoryId, articles, navigate }) {
  const [main, ...subs] = articles;
  return (
    <section className="section-block">
      <div className="section-header">
        <h2>{title}</h2>
        <a href={`#/kategorie/${categoryId}`}
          onClick={e => { e.preventDefault(); navigate(`/kategorie/${categoryId}`); }}
          className="see-all">
          Vše z {title} →
        </a>
      </div>
      <div className="section-grid">
        <div className="main-article">
          {main && <ArticleCard article={main} size="large" navigate={navigate} />}
        </div>
        <div className="sub-articles">
          {subs.map(a => <ArticleCard key={a.id} article={a} size="small" navigate={navigate} />)}
        </div>
      </div>
    </section>
  );
}
```

6.7 frontend/src/utils.js + config.js

```
// utils.js
export function formatTimeAgo(iso) {
  if (!iso) return '';
  const diff = Date.now() - new Date(iso).getTime();
  const mins = Math.floor(diff / 60000);
  const hours = Math.floor(mins / 60);
  const days = Math.floor(hours / 24);
  if (mins < 2) return 'právě teď';
  if (mins < 60) return `před ${mins} min`;
  if (hours < 24) return `před ${hours} h`;
  if (days < 7) return `před ${days} dní`;
  return new Date(iso).toLocaleDateString('cs-CZ',
    { day: 'numeric', month: 'long', year: 'numeric' });
}

export function formatDate(iso) {
  if (!iso) return '';
  return new Date(iso).toLocaleDateString('cs-CZ',
    { day: 'numeric', month: 'long', year: 'numeric', hour: '2-digit', minute: '2-digit' });
}

// config.js
export const CATEGORIES = [
  { id: 'domaci', label: 'Domácí' },
  { id: 'zahranicni', label: 'Zahraničí' },
  { id: 'ekonomika', label: 'Ekonomika' },
  { id: 'sport', label: 'Sport' },
  { id: 'kultura', label: 'Kultura' },
  { id: 'technologie', label: 'Technologie' },
  { id: 'veda', label: 'Věda' },
  { id: 'zdravi', label: 'Zdraví' },
  { id: 'cestovani', label: 'Cestování' },
  { id: 'krimi', label: 'Krimi' },
];
```


7. KONFIGURACE

backend/package.json

```
{
  "name": "news-portal",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node backend/server.js",
    "dev": "node --watch backend/server.js"
  },
  "dependencies": {
    "@anthropic-ai/sdk": "^0.51.0",
    "axios": "^1.6.0",
    "better-sqlite3": "^9.4.0",
    "cheerio": "^1.0.0",
    "cors": "^2.8.5",
    "express": "^4.18.0",
    "node-cron": "^3.0.3",
    "rss-parser": "^3.13.0"
  }
}
```

frontend/package.json

```
{
  "name": "news-portal-frontend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.2.0",
    "vite": "^5.0.0"
  }
}
```

frontend/vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: { '/api': 'http://localhost:3001' } // dev proxy na backend
  }
});
```

.env.example

```
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxxxxxxxxxxxxxxx
PORT=3001
NODE_ENV=development
```

8. INSTALACE A SPUŠTĚNÍ

Krok za krokem

```
# 1. Vytvoř projekt ve VS Code
mkdir news-portal && cd news-portal
code . # otevři ve VS Code

# 2. Zkopíruj strukturu souborů dle souborového stromu (sekce 4)

# 3. Nainstaluj backend závislosti (v rootu projektu)
npm install

# 4. Nainstaluj frontend závislosti
cd frontend && npm install && cd ..

# 5. Nastav API klíč
cp .env.example .env
# Otevři .env a vlož svůj ANTHROPIC_API_KEY

# 6. Vytvoř složku pro databázi
mkdir data

# 7a. Development – spusíš backend (Terminál 1)
node backend/server.js
# → Server běží na http://localhost:3001
# → Za 5s automaticky spustí první pipeline (RSS fetch + scraping + AI)

# 7b. Development – spusíš frontend (Terminál 2)
cd frontend && npm run dev
# → Vite dev server na http://localhost:5173

# 8. Produkční build
cd frontend && npm run build && cd ..
node backend/server.js
# → Vše na http://localhost:3001
```

Verifikace — co by se mělo stát

- Po spuštění backendu: konzole zobrazí "[PIPELINE] Start ..."
- Za ~30s: "novinky: X nových článků" pro každý zdroj
- Za ~2 min: "[AI] ✓ Přepsáno: ..." pro první články
- API test: curl http://localhost:3001/api/stats → {"total":N,"done":N,...}
- Frontend: http://localhost:5173 zobrazí articles grid
- Stats bar nahoře: "Celkem: X | AI přepsáno: Y | Čeká: Z"

9. PRÁVNÍ POZNÁMKY

Aktivita	Právní status	Poznámka
■tení RSS feed■	■ Legální	Weby ho publikují pro tento účel
Scraping fulltextu	■ Šedá zóna	Porušuje ToS, technicky možné
AI p■epis fakt■	■ Fakta jsou volná	P■epis ≠ kopírování
Zve■ejn■ní bez atribuce	■ Rizikové	Lepší vždy uvést zdroj
Odkaz na originál	■ Doporu■eno	Zvyšuje d■iv■ryhodnost i SEO

■ **D■LEŽITÉ:** Vždy zobrazuj odkaz na originální ■lánek ("Zdroj: Novinky.cz"). Implementace v ArticlePage.jsx obsahuje source-attribution sekci. Mezi requesty dodržuj minimálně 2s pauzu (sleep v scraper.js).

■ Bezpe■ná alternativa: Používej pouze RSS perex (bez scrapingu fulltextu) a nech Claude p■epsat/rozvinout perex. Fakta z perexu jsou dosta■ující pro generování hodnotného obsahu.

Kompletní zadání p■ipraveno pro Claude Sonnet 4.6 ve VS Code.
Vygenerováno 08. 03. 2026 14:33